Dataset	Learning Rate	Warmup Steps	Finetuning Steps	Batch Size
SQuAD	1.00E-04	400	40000	16
GSM8k	1.00E-04	75	7500	16
CommonSenseQA	5.00E-05	100	6000	16
LAMBADA	5.00E-05	40	1400	16
WebQuestions	5.00E-04	200	2000	16
NaturalQuestions	5.00E-04	100	5000	256
CoQA	1.00E-04	75	3500	16
PhysicalIQA	1.00E-06	50	600	32
HellaSwag	1.00E-06	100	1000	16

Table 4: Downstream finetuning hyperparameters for the 130M model.

Model	130M	1B
Parameters	136,237,056	1,345,085,440
Transformer Layers	12	24
Attention Heads	12	32
Embedding Dimension	768	2048
Hidden Dimension	3072	8092

Table 5: Architecture details for the models considered in this work

Algorithm 1: Pause-pretraining

Pretraining with Pause

Inputs: Pretraining dataset $\mathcal{D}_{\text{pt}}$, decoder-only model f_θ, number of <pause> tokens M_{pt} to insert, pause token <pause>

$\mathbf{p}_{1:N} \sim \mathcal{D}_{\text{pt}}$ /* Input Sequence from corpus */

$\tilde{\mathbf{p}}_{1:N+M_{pt}} = \texttt{random_insert}(\mathbf{p}_{1:N}, \texttt{<pause>}, M_{\text{pt}})$ /* Insert M_{pt} pause tokens randomly in the original input sequence $\mathbf{p}_{1:N}$, extending its length by M_{pt} */

$S_{\text{ignore}} = \{k \in [0, N + M_{pt} - 1] \colon \tilde{p}_{k+1} = \texttt{<pause>}\}$ /* Identify the set of positions where the next token is <pause> */

$\mathcal{L}_{\text{PausePT}}(f_\theta, \tilde{\mathbf{p}}_{1:N+M_{\text{pt}}}) = \sum_{k=1, k \notin S_{\text{ignore}}}^{N+M_{\text{pt}}-1} \mathcal{L}_{\text{CE}}(\tilde{p}_{k+1}, f_\theta(\tilde{\mathbf{p}}_{1:k}))$ /* Next token prediction error excludes targets which are pause (model isn't made to learn to predict pause itself) */

$\theta = \theta - \nabla_\theta \mathcal{L}_{\text{PausePT}}(f_\theta, \tilde{\mathbf{p}}_{1:N+M_{\text{pt}}})$ /* Update the model */

Algorithm 2: Pause-finetuning

Stage 2: Finetuning with Pause

Inputs: Downstream labeled dataset $\mathcal{D}_{\text{ft}}$, pretrained model f_θ, number of <pause> tokens M_{ft} to insert, pause token <pause>

$\mathbf{p}_{1:N}, \mathbf{t}_{1:T} \sim \mathcal{D}_{\text{ft}}$ /* Sample prefix and target */

$\tilde{\mathbf{p}}_{1:N+M_{\text{ft}}} = \text{Concat}[\mathbf{p}_{1:N}, [\texttt{<pause>}] \times M_{\text{ft}}]$ /* Append prefix and M_{ft} pauses */

$\mathcal{L}_{\text{PauseFT}}(f_\theta, \tilde{\mathbf{p}}_{1:N+M_{\text{ft}}}, \mathbf{t}_{1:T}) = \sum_{k=0}^{T-1} \mathcal{L}_{\text{CE}}(t_{k+1}, f_\theta(\text{Concat}[\tilde{\mathbf{p}}_{1:N+M_{\text{ft}}}, \mathbf{t}_{1:k}])$ /* Next token prediction error on targets */

$\theta = \theta - \nabla_\theta \mathcal{L}_{\text{PauseFT}}(f_\theta, \tilde{\mathbf{p}}_{1:N+M_{\text{ft}}}, \mathbf{t}_{1:T})$

Algorithm 3: Pause-inference

Stage 3: Inference with Pause

Inputs: Prefix $\mathbf{p}_{1:N}$, finetuned model f_θ, number of <pause> tokens M_{inf} to insert, Pause token <pause>

$\tilde{\mathbf{p}}_{[1:N+M_{\text{inf}}]} = [\mathbf{p}_{1:N}, [\texttt{<pause>}] \times M_{\text{inf}}]$ /* Append M_{inf} pauses to prefix */

$\tilde{\mathbf{p}}_{N+M_{\text{inf}}+1} \sim f_\theta(\tilde{\mathbf{p}}_{1:N+M_{\text{inf}}})$ /* Predict the next token in the sequence, and continue in an auto-regressive fashion */

I INFERENCE TIME COST OF PAUSE TOKENS

One way to assess the inference-time compute efficiency of a method is by estimating the number of *Floating Point Operations per Second (FLOPS)* it requires. A related, but independent metric, is the *Wall Clock Time* as it affects the latency of deployed systems. Below, we analyze how efficient pause-inference is along these two metrics in this section. Broadly, we make two arguments:

1. Pause-inference offers a more FLOPS-efficient way of increasing performance, as compared to other natural ways of expanding the number of attention operations in a Transformer, such as by adding layers or by adding attention heads.
2. Pause-inference is also wall-clock-efficient compared to the above techniques as it virtually introduces no overhead. When compared to CoT, pause-inference provides a computationally more granular and cheaper way to improve performance (although still upper-bounded by CoT in terms of *performance*).

I.1 PAUSE TOKENS ALLOW FOR A MORE EFFICIENT USE OF FLOPS

We frame our FLOPS-efficiency analysis as follows. Consider introducing p pause tokens during inference in a given Transformer. How many additional FLOPS does this require? If we spent the same budget of additional FLOPS to introduce more attention operations via other techniques — namely, via an appropriate number of additional layers or additional attention heads — would we find a similar improvement in quality, in terms of metrics like accuracy?

Concretely, we use a running example of the 1B model used in this paper, whose architecture details are provided in Table 5. Specifically, we have number of transformer layers as $l = 24$, input embedding dimension as $h = 2048$, per attention head embedding dimension $d = h/a$, where $a = 32$ is the number of attention heads. We also anchor our analysis for input prompts with $n = 100$ tokens, which represents the average prompt length for many tasks considered in this work. For our analysis, we rely on supporting lemmas deferred to Section I.2.1.

FLOPS-efficiency of p pause tokens vs k additional layers: In the context of the downstream task of SQuAD, appending $p = 10$ pause tokens yields an 18% increase in EM score. Applying these values ($n = 100, p = 10$) and $l = 24$ in the FLOPS-efficiency analysis of Theorem I.3, we can deduce that if we were to allocate the same FLOPS budget to adding more layers to the Transformer stack, we can at most add 2 layers. This enhancement corresponds to a modest 10% rise in parameter count, expanding the model from a 1B parameter model to a 1.1B parameter model. However, in practice, when scaling the parameter count, significant performance improvements (such an 18% increase in EM score) are typically observed only when the parameter count is scaled by much larger factors. Thus we argue that in this case, pause tokens provide a more inference-time-FLOPS-efficient way of increasing performance. Conceptually, this underscores the fact that pause tokens introduce an alternative dimension to representation capacity, distinct from the traditional approach of scaling the parameter count.

Comparing FLOPS with increase in attention heads: In the standard Transformer implementation used in practice, when one increases the attention heads (a), although the number of attention operations increases, the per-attention-head embedding dimension (d) proportionally gets reduced to keep the overall embedding dimension constant (h). Thus, there is effectively no change in the number of FLOPS. In contrast, adding pause tokens increases the number of attention operations, while keeping the per-attention-head embedding dimension d constant. Therefore, for a fair comparison, we consider the case where we increase a, while keeping the per-attention-head dimension d fixed (and we fix it to be equal to h).

Then, from our analysis in Theorem I.4 we have that for an input of length $n = 100$ and $a = 32$ attention heads, appending $p = 10$ pause tokens is equivalent to increasing the number of attention heads by $k = 3$. However, increase the attention heads by 3 in the 1B model configuration adds only 48M parameters (we add $W_q, W_k, W_v, W_{proj} \in \mathbb{R}^{h \times h}$ per additional attention head), bringing the model to a parameter count of 1.048B from 1B. This, we argue cannot account for any significant performance improvement equivalent to the improvements seen under pause-training proposed in this paper.